# MOTIF 빌드·배포 매뉴얼

기준일: 2026-09-28 (master `3c98bcd1`). "실측" 으로 표기한 값은 운영 서버에서 직접 확인한 것이고, 나머지는 레포의 매니페스트·빌드 스크립트 기준이다.

## 1. 시스템 개요

MOTIF 는 **안드로이드 앱 + Spring Boot 4 멀티모듈 백엔드 + GPU AI 합성 파이프라인**으로 구성된다. 백엔드는 SSAFY 지급 서버 1대(4 vCPU / 15GB RAM / 309GB, Ubuntu 24.04.4) 위의 **단일 노드 k3s** 에 배포되고, AI 합성은 GCP Cloud Run(GPU) 에서 돌며, 영상·이미지는 AWS S3 + CloudFront 로 전달한다.

```
사용자(안드로이드 앱)
   │ HTTPS
   ▼
Traefik (k3s 내장 · hostPort 80/443 · TLS 종단, Let's Encrypt)
   │
   ├ /api/**     ──▶ core-api (Spring :8081) ──▶ PostgreSQL · Redis
   │                    ├ MQ job.media ─────────▶ app-media (ffmpeg → HLS·썸네일 → S3)
   │                    ├ MQ evt.notification ▶ app-notification ──▶ FCM 푸시
   │                    └ HTTP 접수 ──────────▶ AI 접수(GCP Cloud Run) ──▶ GPU Job
   │                         ◀─ 콜백 /api/worker/v1/composite-jobs/{uuid}/done ┘
   ├ /admin/**   ──▶ core-api (테일넷 대역만 허용, 그 외 404)
   └ /project/** ──▶ Jenkins (호스트 프로세스, GitLab 웹훅 전용)

미디어: 앱 ──▶ S3 presigned PUT(업로드) · CloudFront(공개 URL, private/* 는 서명 URL) ──▶ 앱
```

- HTTP 를 받는 서비스는 **core-api 하나**다. app-media·app-notification 은 RabbitMQ 컨슈머라 Service·Ingress 가 없다. 게이트웨이는 두지 않는다(ADR-0009).
- Traefik 은 hostPort 80/443 직결이라 klipper-lb 의 SNAT 이 없고, 클라이언트 원 IP 가 앱까지 도달한다. 어드민 IP 필터가 이 전제 위에 있다.
- 인증은 core-api 의 Spring Security 가 담당한다. 구글 ID 토큰을 JWKS 로 검증한 뒤 자체 JWT(HS256) 를 발급한다.
- AI 쪽에는 DB·큐·상태가 없다. 잡 상태·재시도·HLS 가공·푸시는 전부 백엔드가 관리하고, AI 는 접수 → GPU Job 1회 실행 → S3 PUT → 콜백 1회로 끝난다.
- AI 접수 URL(`AI_SUBMIT_URL`)이 비어 있으면 디스패처는 접수를 건너뛰고 잡을 QUEUED 로 유지한다. 시연에서는 `backend/tools/demo/demo_complete.py` 로 강제 완료할 수 있다(§9).

## 2. 기술 스택 및 버전

| 구성 | 버전 | 비고 |
|---|---|---|
| 서버 OS | Ubuntu 24.04.4 LTS (커널 6.17.0-1019-aws), 4 vCPU / 15Gi RAM / 309GB (실측) | SSAFY 지급 서버, 스왑 0 |
| k3s | v1.36.4+k3s1, containerd 2.3.4 (실측) | 단일 노드. Traefik v3 · local-path 스토리지 내장 |

| 구성 | 버전 | 비고 |
|---|---|---|
| cert-manager | v1.16.2 | Let's Encrypt http01, ClusterIssuer `letsencrypt-prod` |
| Java (JVM) | 21 — 컨테이너 `eclipse-temurin:21-jre`, 호스트 OpenJDK 21.0.12.1 (실측) | Gradle toolchain 21 (foojay 자동 프로비저닝) |
| Spring Boot | 4.0.8 | `backend/gradle/libs.versions.toml` |
| Gradle (백엔드) | 9.7.1 (wrapper, Groovy DSL) | 버전 카탈로그 |
| 백엔드 주요 라이브러리 | Spring Security + oauth2-jose, jjwt 0.13.0, Data JPA + QueryDSL 7.6(openfeign), Liquibase(Boot BOM), Spring AMQP, Data Redis, AWS SDK v2 2.32.14 (s3·cloudfront), firebase-admin 9.4.3, springdoc 3.0.3, MapStruct 1.6.3, Lombok 1.18.46 | |
| PostgreSQL | 16.15 (`postgres:16-alpine`, 실측) | 확장 pg_trgm 1.6 |
| RabbitMQ | 3.13.7 (`rabbitmq:3.13-management`, 실측) | vhost `/motif`, 큐 5개 + DLQ 5개 |
| Redis | 7.4.11 (`redis:7-alpine`, 실측) | 무영속(`save ""`, `appendonly no`), maxmemory 200mb |
| ffmpeg | app-media 이미지에 apk 설치(버전 미고정) | 렌디션·HLS·썸네일 |
| Jenkins | 2.568.2 LTS (실측, 호스트 systemd) | 잡 `motif-core-api` 1개 |
| Docker | 29.1.3 (실측, `docker.io`) | 이미지 빌드 전용. 런타임은 containerd |
| Tailscale | install.sh 최신 | 노드 테일넷 IP 100.106.64.43 (`motif-server`) |
| 안드로이드 | Kotlin 2.3.21, AGP 9.2.1, Gradle 9.5.0, Compose BOM 2026.02.01, Navigation 3 1.1.3, Hilt 2.59.2, compileSdk 37 / minSdk 26 / targetSdk 37, Java 17 | `android/gradle/libs.versions.toml` |
| AI 파이프라인 | Python 3.12 (uv 0.11.28) + 모델별 3.10 격리 env, PyTorch 2.7.0 (cu128), 베이스 `nvidia/cuda:12.8.1-cudnn-devel-ubuntu22.04`, Cloud Run Jobs GPU `nvidia-rtx-pro-6000` (96GB) | `ai_pipeline/Dockerfile`, `requirements.txt` |
| IDE | IntelliJ IDEA(백엔드), Android Studio(앱) | |

### 2.1 앱 컨테이너 이미지

| 이미지 | Dockerfile | 베이스 | 내용 |
|---|---|---|---|
| `registry.local/motif/core-api:<빌드번호>` | `backend/app-core-api/Dockerfile` | `eclipse-temurin:21-jre` | 레이어드 jar(ADR-0008), 비루트 uid 1001 |

| 이미지 | Dockerfile | 베이스 | 내용 |
|---|---|---|---|
| `registry.local/motif/app-media:<빌드번호>` | `backend/app-media/Dockerfile` | `eclipse-temurin:21-jre-alpine` + ffmpeg | 동일 |
| `registry.local/motif/app-notification:<빌드번호>` | `backend/app-notification/Dockerfile` | `eclipse-temurin:21-jre-alpine` | 동일 |
| `us-central1-docker.pkg.dev/ssafy-motif/video-workers/motif-compose:<태그>` | `ai_pipeline/Dockerfile` | `nvidia/cuda:12.8.1-cudnn-devel-ubuntu22.04` | GPU 합성·사전분석 공용 이미지(가중치는 GCS 마운트, 이미지에 포함 안 함). 실측 배포 태그 v32(합성)·v31(사전분석) |
| `…/video-workers/motif-intake:<태그>` | `ai_pipeline/Dockerfile.intake` | `python:3.12-slim` | 접수 서비스(CPU). 실측 v3 |

`registry.local` 은 실재하지 않는 프리픽스다. 백엔드는 레지스트리 없이 Jenkins 호스트에서 `docker build` → `docker save` → `k3s ctr images import` 로 노드 containerd 에 직접 넣고(ADR-0007), `imagePullPolicy: Never` 로 임포트 누락을 즉시 드러낸다. 태그는 Jenkins 빌드번호이며 `latest` 는 쓰지 않는다. 실측 현재 태그 **186**, 모듈별 최근 5개만 보관. AI 이미지는 GCP Artifact Registry 에 둔다(§11).

## 3. 저장소·서버 디렉터리 구조

레포 `infra/k8s/` 가 인프라 구성의 단일 진실원이다. 서버에서 `kubectl edit` 로 고치지 않고 매니페스트를 고쳐 apply 한다.

| 위치 | 내용 |
|---|---|
| (레포) `backend/` | Gradle 멀티모듈. `module-common ← module-event ← (domain-core, domain-notif) ← app-*`. 실행체 `app-core-api` (:8081) · `app-media` (:8083) · `app-notification` (:8082) |
| (레포) `backend/Jenkinsfile` | CI/CD 파이프라인 정의(잡 `motif-core-api` ) |
| (레포) `backend/local-infra/` | 로컬 compose(PostgreSQL·RabbitMQ·Redis·스텁 AI 접수 서비스) |
| (레포) `backend/docs/` | ADR( `adr/` ), API·어드민·AI 명세( `api/` ), DB 명세( `erd/` ), PoC( `poc/` ) |
| (레포) `backend/tools/demo/` | 시연용 합성 강제 완료 스크립트 |

| 위치 | 내용 |
| --- | --- |
| (레포) `infra/k8s/` | k8s 매니페스트. 접두 번호가 적용 순서: `00-cert-manager` `10-postgres` `20-rabbitmq` `30-redis` `40-core-api` `45-app-media` `46-app-notification` `50-jenkins-webhook` `90-ingress` |
| (레포) `infra/setup/` | Jenkins 설치·웹훅 전환·CDN 설정 절차 |
| (레포) `infra/README.md` | 서버 초기 구축 절차(Tailscale → k3s → cert-manager → 순서 apply) |
| (레포) `ai_pipeline/` | AI 합성 파이프라인(Python). `Dockerfile` · `cloudbuild*.yaml`, 운영 문서 `docs/01~16`, vendored 모델 `vendor/` |
| (레포) `android/` | 안드로이드 앱(Gradle 멀티모듈, Compose, `feature/*/api·impl` 구조) |
| (레포) `exec/` | 제출 문서·DB 덤프(이 폴더) |
| (서버) `~/motif` | 소스 체크아웃. 빌드는 `develop/backend`, 매니페스트는 `develop/infra` 기준 |
| (서버) `/etc/rancher/k3s/k3s.yaml` | kubeconfig(644). Jenkins 전역 환경변수 `KUBECONFIG` 도 이 경로 |
| (서버) `/var/lib/rancher/k3s/storage/` | local-path PV 실체(postgres-pvc 10Gi, rabbitmq-pvc 5Gi, core-api-audit-pvc 1Gi) |
| (서버) `/var/lib/jenkins/` | Jenkins 홈. `.gradle/gradle.properties` (데몬 힙), `.profile` (KUBECONFIG) |
| (서버) `/etc/systemd/system/jenkins.service.d/override.conf` | Jenkins 바인딩(테일넷 IP:8080)·힙 1g |
| (서버) `/etc/sudoers.d/jenkins-k3s` | jenkins 계정 NOPASSWD — `k3s ctr images import/ls/rm` 3종만 |
| (서버) `~/demo_complete.py`, `~/motif-reset.sh` | 시연 강제 완료·초기화 스크립트 사본 |
| (GCP) 프로젝트 `ssafy-motif` | Cloud Run Service `motif-intake` (asia-southeast1), Cloud Run Jobs `motif-compose-job` · `motif-precompute-job` (us-central1), Artifact Registry `video-workers`, GCS `ssafy-motif-models-us` (가중치), Secret Manager 3종 |

## 4. 환경 변수

값은 전부 k8s Secret/ConfigMap(백엔드) 과 GCP Secret Manager·Cloud Run env(AI) 안에만 존재하며 레포에는 키 목록( `secret.example.yaml.txt` )만 있다. 전체 키·용도는 `05_환경변수_전체목록` 참조. 요약:

| Secret / ConfigMap | 키 수 (실측) | 소비 주체 |
|---|---|---|
| `postgres-secret` | 3 | PostgreSQL 초기화(superuser·core_app·notif_app 비밀번호) |
| `rabbitmq-secret` | 5 | RabbitMQ 계정 5개(admin·core-api·notification·gpu-worker·media) |
| `redis-secret` | 1 | Redis requirepass |
| `core-api-secret` | 12 | core-api — DB·MQ·Redis 비밀번호, JWT 2종, Google 클라이언트 ID, AWS 키, CloudFront 개인키, 워커 토큰, 어드민 시드 해시, 토스 키 |
| `core-api-config` | 2 | core-api — `ADMIN_ALLOWED_CIDRS`, `AI_SUBMIT_URL` |
| `app-media-secret` | 4 | app-media — DB·MQ 비밀번호, AWS 키 |
| `app-notification-secret` | 3 | app-notification — DB·MQ·Redis 비밀번호 |
| `app-notification-fcm` | 1 (파일) | app-notification — Firebase 서비스 계정 JSON (`/etc/fcm/service-account.json`) |
| `motif-tls` | 2 | Traefik TLS(cert-manager 가 관리) |
| (GCP) Secret Manager `motif-worker-token`, `aws-access-key-id`, `aws-secret-access-key` | 3 | Cloud Run Jobs·intake — 워커 토큰(백엔드 `MOTIF_WORKER_TOKEN` 과 같은 값), IAM `motif-gpu-worker` 키 |
| (앱) `local.properties`, `google-services.json` | 2 파일 | 안드로이드 빌드 — 토스 클라이언트 키, Firebase 설정 |

Secret 을 바꾼 뒤에는 반드시 `kubectl rollout restart deployment/<name>` 을 한다. `envFrom` 은 파드 생성 시 한 번만 읽는다.

## 5. 데이터베이스 구성

단일 PostgreSQL 16 인스턴스(`deployment/postgres`) 안에 DB 1개, 스키마 2개, 계정 2개로 소유권을 나눈다.

| DB | 스키마 | 소유 계정 | 사용 서버 | 스키마 관리 | 테이블 (실측) |
|---|---|---|---|---|---|
| `motif` | `motif_core` | `core_app` | core-api, app-media | Liquibase (core-api 기동 시, changelog 22개) | 41 |
| `motif` | `motif_notif` | `notif_app` | app-notification | Liquibase (app-notification 기동 시) | 3 |

- 초기화: ConfigMap `postgres-init` 의 `init.sh` 가 **PVC 가 비어 있을 때 1회** 실행된다 — `pg_trgm` 확장, 스키마·계정 생성, 교차 접근 REVOKE, `search_path` 설정. 다시 돌리려면 PVC 를 지워야 한다(데이터 소실).
- 스키마 마이그레이션: `spring.jpa.hibernate.ddl-auto: validate` 고정, Liquibase 가 각 스키마 소유 계정으로 기동 시 실행. `databasechangelog` 도 자기 스키마에 둔다. core-api 의 startupProbe 150초는 Liquibase 예산이다.

- 데이터 볼륨: `postgres-pvc` 10Gi(local-path → 노드 로컬 디스크). 노드 소실 = 데이터 소실. S3 백업 크론은 미구현(후속).
- 접근: 테일넷 `100.106.64.43:5432` 만(Service externalIPs). 공인망 미노출.
- 실측: PostgreSQL 16.15, pg_trgm 1.6, 테이블 44개, DB 크기 11MB. 덤프·복원 절차는 `03_README.md`.

## 6. 포트 맵 (실측)

| 구분 | 포트 | 서비스 |
|---|---|---|
| 공인망 노출 | 22 | SSH (ufw. **절대 건드리지 않는다**) |
| | 80 / 443 | Traefik hostPort (80 은 https 리다이렉트). `/api` `/admin` `/project` `/swagger-ui` `/v3/api-docs` 만 라우팅 |
| 테일넷 전용 (100.106.64.43) | 5432 | PostgreSQL (Service `postgres` externalIPs) |
| | 5672 / 15672 | RabbitMQ AMQP / 관리 UI (Service `rabbitmq-tailnet`) — GPU 워커·팀원 |
| | 8080 | Jenkins UI (호스트 바인딩) |
| | 6443 | k3s API 서버 |
| 클러스터 내부 (ClusterIP) | 8081 | core-api (유일한 HTTP 서비스) |
| | 5432 / 5672 / 6379 | postgres / rabbitmq / redis (Service 이름으로 접근) |
| | 8080 | `jenkins-host` (셀렉터 없는 Service + EndpointSlice → 호스트 Jenkins, 웹훅 경로용) |
| 파드 내부만 | 9081 / 9082 / 9083 | core-api / app-notification / app-media 관리 포트(actuator health·prometheus). Service 없음, 프로브 전용 |
| | 8082 / 8083 | app-notification / app-media 서버 포트. 아무도 호출하지 않는다 |
| (GCP) | 443 | `motif-intake` Cloud Run URL (allUsers 허용, 토큰 헤더로 인증) |

방화벽: ufw 22/80/443 만 개방. k3s 는 NodePort 를 쓰지 않는다(ufw 우회 방지).

## 7. 로컬 개발 환경 (백엔드)

프로파일로 3가지 모드를 고른다(ADR-0003).

| 모드 | 프로파일 | DB | MQ · Redis | 용도 |
|---|---|---|---|---|
| 격리 | `local` (기본) | compose | compose | 기능 개발 |
| 하이브리드 | `local-remote` | compose | MQ 는 서버(테일넷), Redis 는 로컬 | 파이프라인 통합 확인 |
| 서버 | `prod` | 클러스터 | 클러스터 | k8s 전용 |

```
# 인프라 기동 (PostgreSQL + RabbitMQ + Redis + 스텁 AI 접수 서비스)
cd backend/local-infra && docker compose up -d --wait
#   PG localhost:5432 (motif/motif123, 앱 계정 core_app·notif_app / motif123)
#   MQ localhost:5672, 관리 UI http://localhost:15672 (motif/motif)   Redis localhost:6379 (motif123)
#   stub-worker http://localhost:18080     재초기화: docker compose down -v

# 실행 (프로파일 미지정 = local)
cd backend && ./gradlew :app-core-api:bootRun
cd backend && ./gradlew :app-notification:bootRun
cd backend && ./gradlew :app-media:bootRun        # ffmpeg 가 PATH 에 있어야 한다

# 빌드 · 테스트
cd backend && ./gradlew :app-core-api:bootJar :app-media:bootJar :app-notification:bootJar
cd backend && ./gradlew :app-core-api:test :app-notification:test :app-media:test   # 외부 의존 없음
cd backend && ./gradlew :app-core-api:localDbTest                                    # compose 필요

# 확인
curl localhost:8081/api/health       # {"status":"UP"}
```

- 필요 도구: JDK 21(없으면 Gradle 이 자동 설치), Docker + compose v2, Python 3(스모크·tools), Tailscale(하이브리드), ffmpeg(app-media).
- 구글 로그인 없이 토큰을 얻으려면 local 전용 `POST /api/dev/auth/login {"oauthSubject":"dev-1"}` 를 쓴다(운영에는 없음).
- 로컬 어드민: `http://localhost:8081/admin/login`, `admin` / `admin1234` (로컬 시드 더미).
- 로컬 S3 는 LocalStack 대신 실버킷 `dev/` 프리픽스를 쓴다 — AWS 키가 필요하다.

## 8. CI/CD (백엔드)

### 8.1 Jenkins

- 잡 1개 `motif-core-api` (Pipeline script from SCM): Git `https://lab.ssafy.com/s15-ai-image-sub1/S15P21D104.git`, Branch `*/develop/backend`, Script Path `backend/Jenkinsfile`, Lightweight checkout 해제.
- 트리거: **GitLab 웹훅**(Push events, 브랜치 `develop/backend`) → `https://j15d104.p.ssafy.io/project/motif-core-api`. 폴링 없음(2026-09-02 웹훅 전환). 방어선은 GitLab 플러그인 Secret Token.
- Jenkins 는 호스트 systemd 프로세스로 테일넷 IP `100.106.64.43:8080` 에만 바인딩된다. 공인망에는 `/project/` 만 열려 있고 `/login` 등 UI 는 닿지 않는다. UI 는 테일넷 또는 SSH 터널( `ssh -i J15D104T.pem -L 8080:127.0.0.1:8080 ubuntu@j15d104.p.ssafy.io` ).
- 자격증명: `gitlab-pat` (Username with password, PAT `jenkins-motif`, `read_repository`), `gitlab-api` (Project Access Token, `api` — 커밋 상태 보고), 웹훅 Secret Token.
- 잡 파라미터: `FORCE_DEPLOY` (변경 없어도 전 모듈 배포), `DEPLOY_MODULES` (쉼표 구분, 예 `app-media`).
- AI 파이프라인·안드로이드는 Jenkins 대상이 아니다(AI 는 `gcloud builds submit` 수동, 앱은 로컬 빌드).

### 8.2 파이프라인 ( `backend/Jenkinsfile` )

1. **Checkout** — `checkout scm`, GitLab 커밋 상태 `running`

2. **Select modules** — 변경 경로로 배포 대상 판정. `backend/module-*`, `domain-*`, `build.gradle`, `settings.gradle`, `gradle/` 가 바뀌면 전 모듈. `backend/**` 변경이 없으면 전체 스킵
3. **Build & Test** — `./gradlew :app-x:bootJar :app-x:test` (한 번의 Gradle 호출). 기본 테스트는 DB·MQ 불필요 (`local-db` 태그 제외)
4. **Docker Build & Import** — 모듈마다 `docker build -t registry.local/motif/<name>:<BUILD_NUMBER>` → `docker save | sudo k3s ctr images import -` → `ctr images ls` 로 임포트 확인
5. **Deploy** — 현재 태그를 `PREV_IMAGE` 로 저장 → `kubectl set image` → `rollout status --timeout=180s` → core-api 는 `https://j15d104.p.ssafy.io/api/health` 3회 확인. 워커는 readiness(db·rabbit)로 갈음
6. **Cleanup old images** — containerd·docker 양쪽에서 모듈별 최신 5개 초과분 삭제
7. **post** — GitLab 상태 `success` / `failed` (파드·이벤트 20줄 출력)/ `canceled`

옵션: 타임아웃 45분, 동시 빌드 금지, 로그 30개 보관. 실측 웹훅 → 빌드 시작 15초.

### 8.3 무중단 배포와 롤백

- core-api Deployment: `RollingUpdate maxUnavailable 0 / maxSurge 1` + `preStop sleep 10` + `terminationGracePeriodSeconds 45` (graceful shutdown 30s). 실측 0.2초 간격 517회 요청 중 실패 0.
- app-media·app-notification 은 `Recreate` (컨슈머 2개 동시 실행 방지). 배포 중 잠깐 멈추지만 큐가 버퍼링한다.
- 자동 롤백 `rollbackAndFail()` : health 실패 시 `kubectl set image` 로 **직전 태그(PREV_IMAGE)** 로 되돌리고 실패 처리. `rollout undo` 를 쓰지 않는 이유는 연속 실패 시 리비전이 한 칸씩 후퇴한 사고(빌드 29~33) 때문이다.
- 수동 롤백: `kubectl set image deployment/core-api core-api=registry.local/motif/core-api:<이전태그>`. 보관 이미지는 `sudo /usr/local/bin/k3s ctr images ls -q | grep registry.local/motif`.

## 9. 배포·운영 특이사항 (트러블슈팅)

1. **`REPLACE_ME` 태그** — `40-core-api/deployment.yaml` 등 앱 Deployment 의 이미지 태그는 최초 apply 전에 사람이 실제 빌드번호로 치환한다(커밋 금지). 이후 이 파일을 그대로 재-apply 하면 태그가 `REPLACE_ME` 로 돌아가 `ErrImageNeverPull` 이 난다. 다른 필드를 고칠 때는 현재 태그를 먼저 읽어 반영한다.
2. **Secret 없이 매니페스트 apply 금지** — `RABBITMQ_MEDIA_PASSWORD` 가 없으면 RabbitMQ postStart 훅이 죽어 파드가 안 뜬다. `core-api-audit-pvc` 가 없으면 core-api 가 CrashLoopBackOff(감사 로그 어펜더 실패). JWT 비밀 2종은 32자 이상·서로 다른 값(19자로 넣어 CrashLoop 난 사고 있음).
3. **`10-postgres/secret.example.yaml`** 만 확장자가 `.yaml` 이라 `kubectl apply -f infra/k8s/10-postgres/` 로 디렉터리 apply 하면 실제 Secret 이 `<REDACTED>` 로 덮인다(실제 사고). 파일 단위로 apply 한다.
4. **AWS 키·CloudFront 개인키 주입 확인** — 서버 IMDS 가 SSAFY 계정 역할을 흘리므로 `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` 를 반드시 Secret 으로 넣는다. 빠지면 파드는 Ready 인데 S3 호출만 AccessDenied. `CLOUDFRONT_PRIVATE_KEY` 는 base64 patch 로 넣고 `printenv | wc -c` 로 길이(1704)를 확인한다(파일이 없으면 빈 값으로 성공하는 함정). 비면 보호 영상 URL 발급이 500.
5. **Swagger 공개 노출** — `40-core-api/ingress.yaml` 의 `/swagger-ui.html` `/swagger-ui` `/v3/api-docs` 세 블록은 서버에서 API 를 확인하려고 일부러 연 것이다. **심사·발표 전에 지우고 재-apply 하면 닫힌다.** 실측 2026-09-28 현재 열려 있다.
6. **개발 로그인 테스트 페이지** `/api/dev/login-test.html` 은 정적 리소스라 운영에서도 열린다(구글 로그인 확인용, 권한은 없음). 백도어 `POST /api/dev/auth/login` 은 `@Profile("local")` 이라 운영 404.
7. **어드민은 테일넷 전용** — 공인망 `/admin/**` 404 가 정상( `AdminIpFilter`, `ADMIN_ALLOWED_CIDRS=100.64.0.0/10` ). ipAllowList 미들웨어는 붙이지 않는다(403 이 어드민 존재를 드러냄). 접

속하려면 Tailscale 조인 + hosts 에 `100.106.64.43 j15d104.p.ssafy.io` 한 줄. 도메인이 공인 IP 로 풀리므로 테일넷에 붙었다는 것만으로는 안 된다. 끝나면 hosts 줄을 지운다(남으면 `/api/**` 도 테일넷을 탄다).

8. **AI 접수 URL 이 비어 있음** — 합성 잡이 QUEUED 로 남는다. 시연은 운영 노드에서 `python3 backend/tools/demo/demo_complete.py [job_uuid]` 로 가장 최근 미완료 CHALLENGE 잡을 미리 만든 결과 영상으로 DONE 처리한다(FCM 까지 정상 경로). 실제 AI 연동 시에는 `motif-intake` URL 을 `core-api-config` 의 `AI_SUBMIT_URL` 에 넣고 `rollout restart`.
9. **access 토큰 TTL** — 현재 30일(테스트용). 시연 전 `MOTIF_ACCESS_TOKEN_TTL=1h` 를 `core-api-config` 에 넣고 `rollout restart`.
10. **토스 키** — `test_sk_…` 이면 실카드 결제가 안 된다. 라이브 전환은 값 교체 + rollout restart.
11. **Let's Encrypt prod 는 주 5회 실패 제한** — 인증서 설정을 바꿀 때는 `letsencrypt-staging` 으로 먼저 확인한다. `kubectl get certificate motif-tls` 로 READY 확인.
12. **Traefik HelmChartConfig 재적용** 시 80/443 이 수 초 끊긴다. `externalTrafficPolicy: Local` 로 되돌리지 않는다(원 IP 보존 실측 실패).
13. **PVC 삭제 = 데이터 소실.** Redis 는 무영속이지만 랭킹·조회수 버퍼는 재계산 가능하고 refresh 토큰은 DB 에 있다.
14. 메모리 배분 전제(스왑 0): Jenkins 1g, Gradle 2g, core-api 1Gi(배포 중 2Gi), PostgreSQL 1.5Gi, RabbitMQ 640Mi, Redis 300Mi, app-media 1Gi, app-notification 768Mi.

## 10. 신규 환경 구축 순서 (백엔드)

`infra/README.md` 와 `infra/setup/jenkins-setup.md` 가 정본이다. 값은 팀 비밀 저장소에서 가져온다.

1. Tailscale 설치·조인( `tailscale up --hostname=motif-server` , 콘솔에서 key expiry 해제). ufw 22/80/443.
2. k3s 설치: `curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="--flannel-iface=tailscale0 --write-kubeconfig-mode=644" sh -`
3. cert-manager v1.16.2 apply → 파드 3개 Running 확인 → `kubectl apply -f infra/k8s/00-cert-manager/`
4. `kubectl apply -f infra/k8s/90-ingress/` (Traefik hostPort 설정 + https 리다이렉트 미들웨어)
5. `kubectl create secret generic postgres-secret …` (3키) → `10-postgres/` 를 **파일 단위로** apply( `secret.example.yaml` 제외) → 로그에 `ready to accept connections`
6. `rabbitmq-secret` (5키) → `kubectl apply -f infra/k8s/20-rabbitmq/`
7. `redis-secret` → `kubectl apply -f infra/k8s/30-redis/`
8. `core-api-secret` (12키, 예시 파일의 생성 명령 + AWS 키·PEM·워커 토큰·시드 해시·토스 키 patch) → `40-core-api/configmap.yaml` `pvc.yaml` apply
9. 최초 이미지 수동 빌드: `cd ~/motif/backend && ./gradlew :app-core-api:bootJar` → `sudo docker build -t registry.local/motif/core-api:0 backend/app-core-api` → `sudo docker save … | sudo /usr/local/bin/k3s ctr images import -`
10. `deployment.yaml` 의 `REPLACE_ME` 를 `0` 으로 치환해 apply → 클러스터 내부 `curl http://core-api:8081/api/health` = `{"status":"UP"}` → `ingress.yaml` `ingress-admin.yaml` apply → `curl https://j15d104.p.ssafy.io/api/health` → `kubectl get certificate motif-tls` READY
11. `app-media-secret` → `45-app-media/deployment.yaml` (태그 치환) / `app-notification-secret` + `app-notification-fcm` ( `--from-file=service-account.json=…` ) → `46-app-notification/deployment.yaml`. **app-notification 이 떠야 `motif_notif` 스키마와 `device_tokens` 가 생긴다.**
12. Jenkins 셋업(JDK 21 → docker.io → Jenkins LTS → 테일넷 바인딩 → 플러그인 Git·Pipeline·Credentials Binding·GitLab → 자격증명 → sudoers → KUBECONFIG → Gradle 데몬 설정 → 잡 생성) → `50-jenkins-`

`webhook/webhook-route.yaml` apply → GitLab 웹훅 등록 → `Build Now`

13. (기존 데이터 이관 시) `03_README.md` 절차로 DB 덤프 복원, S3 버킷 `motif-media` 객체 동기화, CloudFront 키 그룹 등록
14. AI 접수 서비스 배포(§11) 후 `AI_SUBMIT_URL` 주입, 앱 빌드(§12)

검증:

```
# {"status":"UP"} 이어야 한다
curl -s https://j15d104.p.ssafy.io/api/health
# 공인망 404 · 테일넷 200
curl -s -o /dev/null -w "%{http_code}\n" https://j15d104.p.ssafy.io/admin/login
# core-api, app-media, app-notification, postgres, rabbitmq, redis 6개 Running
kubectl get pods
# READY True
kubectl get certificate motif-tls
```

## 11. AI 파이프라인 빌드·배포 (GCP)

정본은 `ai_pipeline/docs/07-deployment-notes-2026-09-20.md`, `14-dev-log-2026-09-21.md` (부록 A 운영 명령·B 볼륨), `15-dev-log-2026-09-23.md` (최종 배포 상태).

### 11.1 구성

```
core-api ──POST {AI_SUBMIT_URL}/ (X-Motif-Worker-Token)──▶ motif-intake (Cloud Run Service, CPU)
                                                            │ Jobs API :run, env MOTIF_JOB
                                                            ▼
                                          motif-compose-job (Cloud Run GPU Job, us-central1)
                                            S3 GET raw·편집본·사전분석 → 합성 → S3 PUT results/…
                                            → POST /api/worker/v1/composite-jobs/{uuid}/done · /failed
(사전분석) core-api ──POST …/precompute──▶ motif-precompute-job ──▶ S3 precomputed/{c}/{p}/ready.json
```

| 항목 | 값 |
|---|---|
| GCP 프로젝트 | `ssafy-motif` |
| 접수 서비스 | Cloud Run Service `motif-intake`, asia-southeast1, `https://motif-intake-1025574392259.asia-southeast1.run.app`. allUsers invoker 허용, 인증은 `X-Motif-Worker-Token` 헤더. 합성 `POST /`, 사전분석 `POST /precompute` |
| GPU Job | Cloud Run Jobs `motif-compose-job` (합성), `motif-precompute-job` (사전분석), us-central1. GPU `nvidia-rtx-pro-6000` (96GB), `--cpu=20 --memory=80Gi --tasks=1 --max-retries=0 --task-timeout=3600` |
| 이미지 | Artifact Registry `us-central1-docker.pkg.dev/ssafy-motif/video-workers/motif-compose:{tag}` (GPU 공용), `motif-intake:{tag}`. 실측 배포: compose v32, precompute v31, intake v3 |
| 가중치 | GCS `gs://ssafy-motif-models-us` (총 ~34.6GB, 마운트 대상 ~17.7GB) → Job 볼륨 `/mnt/model-weights` (gcsfuse, readonly, `implicit-dirs`). 이미지에는 굽지 않는다 |

| 항목 | 값 |
|---|---|
| 비밀 | Secret Manager `motif-worker-token`, `aws-access-key-id`, `aws-secret-access-key` → `--set-secrets` |
| 서비스 계정 | 기본 compute SA: 가중치 버킷 `storage.objectViewer`, `secretmanager.secretAccessor`, `run.developer` (intake 가 Job 실행) |
| 스토리지 | AWS S3 `motif-media` (ap-northeast-2, cross-region). 읽기 `raw/*` `private/edited/*` `assets/*` `precomputed/*`, 쓰기 `results/*` `precomputed/*` (IAM `motif-gpu-worker`) |
| 빌드 | Cloud Build 수동( `gcloud builds submit --config` ), 트리거 없음. global 풀 `E2_HIGHCPU_32`, timeout 7200s. 첫 빌드 ~40분, 코드만 바뀌면 ~4~6분( `:cache` 인라인 캐시) |

실측 처리 시간(docs/15): CHALLENGE 합성 약 26~28분, MOTIF 약 14분, 사전분석(슬롯당) 약 50분 — GPU Job 하드 상한 60분에 근접.

### 11.2 절차

전제: `gcloud auth login`, `gcloud config set project ssafy-motif`, Cloud Run·Cloud Build·Artifact Registry·Secret Manager·Storage API 활성, 작업 디렉터리 `ai_pipeline/`. 로컬 Docker 빌드는 amd64 CUDA 컴파일 때문에 비현실적이라 Cloud Build 를 쓴다.

```
# 1) 가중치 1회 적재 (SMPL-X 는 라이선스 등록 후 직접 받아 둔다)
PYTHON=/path/to/python HF_TOKEN_FILE=/srv/secrets/hf_token \
  docker/upload_weights_to_gcs.sh gs://ssafy-motif-models-us /path/SMPLX_NEUTRAL.npz

# 2) pytorch3d wheel 1회 빌드 → docker/wheels/ 에 배치 (없으면 메인 빌드가 소스 컴파일로 +25분)
gcloud builds submit --no-source --config=cloudbuild.pytorch3d.yaml --project=ssafy-motif
gsutil cp gs://ssafy-motif-models-us/wheels/pytorch3d-0.7.8-cp310-cp310-linux_x86_64.whl docker/wheels/

# 3) GPU 이미지 빌드 (반드시 --config. --tag 는 --no-cache 가 강제돼 매번 40분)
gcloud builds submit --config=cloudbuild.yaml --substitutions=_TAG=v33 --project=ssafy-motif .

# 4) 접수 서비스 이미지 빌드
gcloud builds submit --config=cloudbuild.intake.yaml --substitutions=_TAG=v4 --project=ssafy-motif .

# 5) GPU Job 이미지 교체 (최초 생성·볼륨 정의는 docs/07 §3, docs/14 부록 B)
gcloud run jobs update motif-compose-job    --image=us-central1-docker.pkg.dev/ssafy-motif/video-worker
s/motif-compose:v33 --region=us-central1 --project=ssafy-motif
gcloud run jobs update motif-precompute-job --image=us-central1-docker.pkg.dev/ssafy-motif/video-worker
s/motif-compose:v33 --region=us-central1 --project=ssafy-motif

# 6) 접수 서비스 배포 — *_JOB_RESOURCE 의 리전은 Job 이 있는 us-central1 (틀리면 전량 503)
gcloud run deploy motif-intake --region=asia-southeast1 --project=ssafy-motif \
  --image=us-central1-docker.pkg.dev/ssafy-motif/video-workers/motif-intake:v4 --allow-unauthenticated \
  --set-secrets=MOTIF_WORKER_TOKEN=motif-worker-token:latest \
  --set-env-vars=COMPOSE_JOB_RESOURCE=projects/ssafy-motif/locations/us-central1/jobs/motif-compose-job,
PRECOMPUTE_JOB_RESOURCE=projects/ssafy-motif/locations/us-central1/jobs/motif-precompute-job
#   출력 URL 을 백엔드 core-api-config 의 AI_SUBMIT_URL 에 넣고 kubectl rollout restart deployment/core-
api

# 7) 확인
gcloud run jobs executions list --job=motif-compose-job --region=us-central1 --limit=5
```

```
gcloud logging read 'resource.type="cloud_run_job" labels."run.googleapis.com/execution_name"="<EXEC>" N
OT textPayload:"GCSFuse"' --project=ssafy-motif --format="value(timestamp,severity,textPayload)"

# 8) 롤백
gcloud run jobs update motif-compose-job --image=…:v32 --region=us-central1
gcloud run services update-traffic motif-intake --to-revisions=<rev>=100 --region=asia-southeast1
```

- Cloud Run 없이 백엔드 구간만 검증하려면 `ai_pipeline/docs/16-integration-test-guide.md` : S3 에 결과 키를 만들고 `/done` 콜백을 curl 로 쏘면 10~30초 뒤 HLS 가 생긴다.
- CPU 유닛테스트: `python -m unittest run_final_cloud_run_job.test_server run_final_cloud_run_job.test_precomputed_jobs` (디렉터리 `ai_pipeline` 을 `run_final_cloud_run_job` 이름으로 심링크).

### 11.3 주의사항

1. Job 은 **항상 exit 0** 이라 콘솔의 `succeeded` 로 성공을 판단하지 않는다. 로그의 `processing failed` / `Traceback` 유무와 콜백( `/done` vs `/failed` )로 판정한다.
2. `--max-retries=0` , `--tasks=1` 필수. 아니면 OOM 시 같은 잡이 반복 실행되고 콜백이 중복된다. 재시도 판단은 백엔드 몫이다.
3. **사전분석 선행** — 합성은 `precomputed/{챌린지}/{슬롯}/ready.json` 이 없으면 `INPUT_NOT_FOUND` 로 실패한다. 새 챌린지 슬롯(편집본)을 등록하면 반드시 `/precompute` 를 먼저 완료한다(슬롯당 약 50분).
4. task-timeout 3600초는 Cloud Run GPU Job 하드 상한이다. 백엔드는 `timeout + 5분` 뒤 미콜백 잡을 실패로 닫는다.
5. 현재 `motif-compose-job` 에 `MOTIF_SKIP_FINGERPRINT=1` (코드·가중치 지문 게이트 우회, 테스트 플래그)이 설정돼 있다. 끄려면 최종 코드로 전 슬롯 재-사전분석이 필요하다.
6. 라이선스: GVHMR 는 비상업·연구 한정, SMPL-X 는 별도 라이선스, RelightVid 는 vendor 에 LICENSE 파일 없음.

## 12. 안드로이드 앱 빌드

| 항목 | 값 |
|---|---|
| 요구 도구 | JDK 17 이상(Gradle 데몬용 JDK 21 은 foojay 가 자동 설치), Android SDK Platform 37 + Build-Tools, Android Studio(AGP 9.x 지원 버전) |
| 기기 | Android 8.0(API 26) 이상, 후면 카메라, Google Play 서비스(Credential Manager·FCM), 인터넷 |
| applicationId | `com.ssafy.motif` , versionCode 1 / versionName 1.0 |
| 빌드 타입 | 기본 `debug` / `release` 만. 난독화·서명 설정 없음(release 는 unsigned) |

필수 로컬 파일(둘 다 gitignore, 레포에 없음):

```
android/local.properties
  sdk.dir=<Android SDK 경로>
  TOSS_CLIENT_KEY=<토스페이먼츠 테스트 클라이언트 키>   # 없으면 빈 문자열로 빌드되고 결제 진입 시 안내 문
구
android/app/google-services.json                    # Firebase 프로젝트의 Android 앱(com.ssafy.motif)
설정. 없으면 빌드 실패
```

Google 로그인의 웹 클라이언트 ID 는 `android/feature/signin/impl/src/main/res/values/strings.xml` 의 `google_web_client_id` 에 있다. 백엔드 `GOOGLE_CLIENT_ID` 와 같은 값이어야 한다.

```
cd android
./gradlew :app:assembleDebug        # → app/build/outputs/apk/debug/app-debug.apk
./gradlew :app:installDebug         # 연결된 기기에 설치
./gradlew :app:assembleRelease      # unsigned. 배포하려면 signingConfigs 추가 또는 apksigner 서명
adb shell pm grant com.ssafy.motif android.permission.CAMERA   # 앱에 CAMERA 런타임 요청 코드가 없어 수동 허용
```

- API 서버 주소는 `android/core/network/src/main/kotlin/com/ssafy/motif/core/network/NetworkConfig.kt` 의 `BASE_URL` ( `https://j15d104.p.ssafy.io/` )과 `API_HOST` (Bearer 를 붙일 호스트) 두 상수다. 다른 서버로 바꾸려면 둘 다 바꾼다.
- `android/app/src/main/kotlin/com/ssafy/motif/MainActivity.kt` 의 `DEMO_MODE` (현재 `true` )가 켜져 있으면 항상 로그인 화면부터 시작하고 로그인·온보딩은 서버 호출 없이 화면만 진행된다. 실제 흐름은 `false` .
- 런타임 권한 요청은 `POST_NOTIFICATIONS` (Android 13+) 하나다. 촬영본은 무음으로 녹화된다( `RECORD_AUDIO` 없음. 서버가 원본 오디오만 쓰므로 무방).